

FINAL PROJECT REPORT

Blood Donation Management System (BDMS)

A Database Management System for Blood Banks in Sargodha, Pakistan
Built using MySQL, Python (Flask), and HTML/CSS

Course: Database Systems

Submitted by:

- | | |
|--------------------------|---------------|
| 1. Zobia Shaheen | BSAI51S25R018 |
| 2. Shumaila Zafar Ranjha | BSAI51S25R028 |
| 3. Rameen Fatima | BSAI51S25R039 |

Submitted to: Dr. Muhammad Ilyas

Department: Computer Science

University: University of Sargodha

Date: June 2026

Abstract

The Blood Donation Management System (BDMS) is a web-based database application developed to manage the operations of blood banks in Sargodha, Pakistan. The system maintains records of blood donors, recipients, blood inventory, hospitals, donation appointments, and blood requests. It provides three separate user roles — Administrator, Donor, and Recipient — each with its own secure login and dedicated interface.

The project applies core database concepts studied in this course: entity-relationship modelling (Chapters 2 and 3), normalization up to Third Normal Form (Chapter 4), physical database design through indexing (Chapter 5), and database security and concurrency control (Chapter 11). The database is implemented in MySQL, the application logic in Python using the Flask framework, and the user interface in HTML and CSS. The result is a fully functional system that demonstrates how theoretical database principles are applied in a practical, real-world application.

1. Introduction

A Database Management System (DBMS) is software that allows users to define, create, store, and manage data in an organized way, while enforcing rules that keep the data accurate, consistent, and secure. Instead of storing information in scattered files, a DBMS keeps it in related tables that can be queried and updated reliably.

Blood banks handle large amounts of critical information: who can donate, which blood groups are available, which hospitals need blood, and which requests are pending. Managing this manually is slow and error prone. The Blood Donation Management System solves this by storing all of this information in a single, well-structured MySQL database and providing an easy interface to manage it.

This report explains the design and implementation of the system and shows clearly where each studied database concept has been applied.

2. Objectives and Scope

2.1 Objectives

- To design a normalized relational database for blood donation management.
- To maintain records of donors, recipients, blood inventory, hospitals, appointments, and requests.
- To implement three secure user roles (Admin, Donor, Recipient).
- To apply database security (password hashing, access control) and concurrency control (transactions and locking).
- To provide a simple, clear web interface for all operations.

2.2 Scope

The current system operates for the city of Sargodha. It covers donor and recipient registration, blood inventory tracking, donation recording, appointment booking, and blood-request approval. Sample data for Sargodha blood banks and hospitals is included.

3. Tools and Technologies

Technology	Purpose
MySQL (via XAMPP)	The relational database that stores all tables and data.
phpMyAdmin	Visual tool used to create and manage the MySQL database.
Python (Flask framework)	The backend that connects the database to the web pages and contains the application logic.
HTML and CSS	The front end that displays the user interface in the browser (light-mode design).
Werkzeug (security library)	Used to hash and verify user passwords securely.
Web browser	Used to access and operate the application at http://127.0.0.1:5000 .

4. System Modules

The system is divided into three role-based modules. After logging in, each user sees only the pages relevant to their role.

4.1 Administrator Module

The administrator manages the entire system. The admin can add and delete donors and recipients, add blood units to inventory, record donations, approve or reject blood requests, view all appointments, and view reports of blood stock by group.

4.2 Donor Module

A donor logs in to a personal portal where they can view their own profile, view their own donation history, and book a donation appointment at a chosen blood bank.

4.3 Recipient Module

A recipient logs in to a personal portal where they can submit a blood request (their blood group is filled in automatically) and track the status of their previous requests.

5. Database Design

5.1 Entities and Attributes

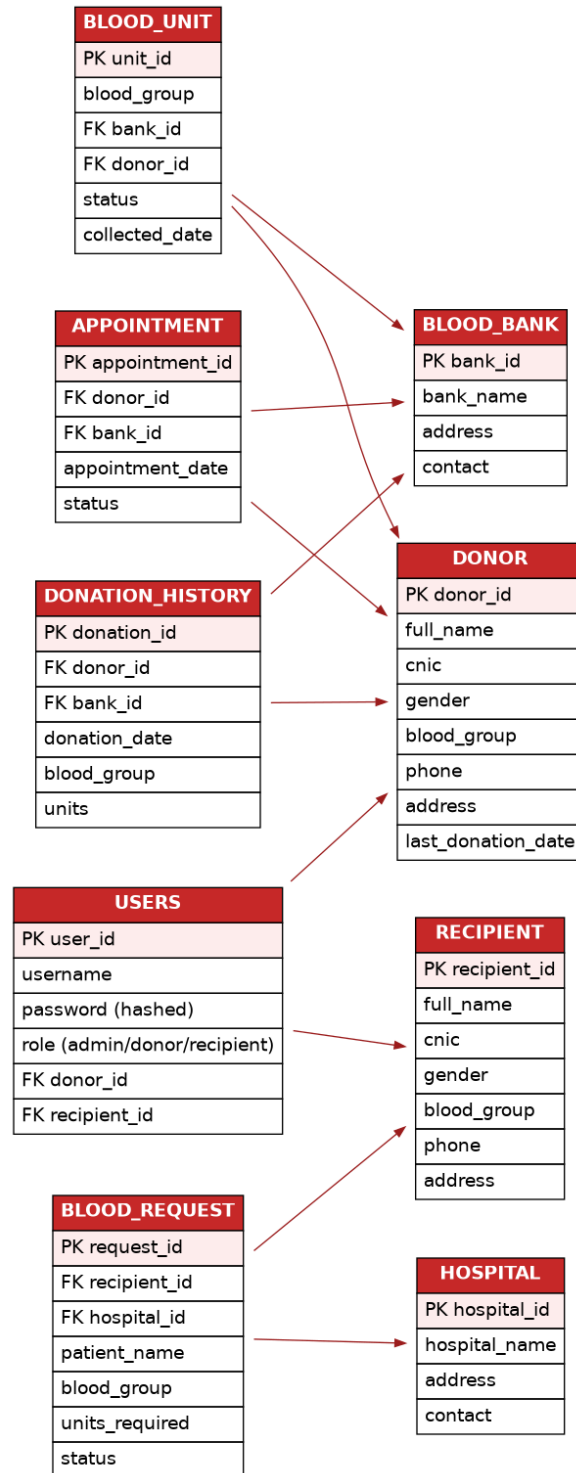
The system is built around nine entities. Each entity becomes a table in the database.

- Donor — people who give blood (name, CNIC, blood group, contact, etc.).
- Recipient — people who need blood.
- Blood Bank — locations where blood is stored and collected.

- Hospital — institutions that request blood for patients.
- Blood Unit — individual units of blood in inventory, with a status.
- Blood Request — a request for blood made by a recipient through a hospital.
- Appointment — a scheduled slot for a donor to donate.
- Donation History — a record of each completed donation.
- Users — login accounts that control access (Admin, Donor, Recipient).

5.2 Entity-Relationship Diagram (Chapters 2 and 3)

The following diagram shows the nine tables, their key attributes, and the foreign-key relationships that connect them. Primary keys are marked PK and foreign keys are marked FK.



Enhanced ER (EER) concepts are also present. The Users entity behaves like a supertype with the role attribute (admin, donor, recipient) acting as the subtype discriminator. Donation History and Appointment act as associative entities linking donors with blood banks, and Blood Request links a recipient with a hospital.

5.3 Relationships

- One donor can have many donations and many appointments (one-to-many).
- One recipient can make many blood requests (one-to-many).
- One blood bank can hold many blood units and host many appointments (one-to-many).
- One hospital can forward many blood requests (one-to-many).
- A user account is linked to one donor or one recipient (one-to-one).

6. Relational Schema

Each entity is implemented as a table with a primary key and, where needed, foreign keys that enforce referential integrity.

Table	Primary Key	Foreign Keys	Purpose
donor	donor_id	—	Stores donor records.
recipient	recipient_id	—	Stores recipient records.
blood_bank	bank_id	—	Stores blood bank locations.
hospital	hospital_id	—	Stores hospital details.
users	user_id	donor_id, recipient_id	Login accounts and roles.
blood_unit	unit_id	bank_id, donor_id	Blood inventory with status.
appointment	appointment_id	donor_id, bank_id	Donation appointments.
blood_request	request_id	recipient_id, hospital_id	Blood requests and status.
donation_history	donation_id	donor_id, bank_id	Completed donation records.

7. Normalization

Normalization is the process of organizing tables to remove redundancy and avoid update problems. All tables in this database are normalized up to Third Normal Form (3NF).

7.1 First Normal Form (1NF)

A table is in 1NF if every column holds a single, atomic value and there are no repeating groups. For example, a donor's blood group is stored as one value, and a donor with multiple donations is handled by separate rows in the donation_history table rather than multiple columns.

7.2 Second Normal Form (2NF)

A table is in 2NF if it is in 1NF and every non-key column depends on the whole primary key. Because each table uses a single-column primary key (such as donor_id), there are no partial dependencies.

7.3 Third Normal Form (3NF)

A table is in 3NF if it is in 2NF and no non-key column depends on another non-key column (no transitive dependency). For example, blood-bank details are not repeated inside the blood_unit table; instead, blood_unit stores only bank_id and the details are kept once in the blood_bank table.

8. Physical Database Design

Physical design decides how the data is actually stored and accessed efficiently. Appropriate data types were chosen for each column (for example, ENUM for blood group and status, DATE for dates, and VARCHAR for names).

9. Database Security

Several security measures protect the system and its data:

9.1 Password Hashing

User passwords are never stored as plain text. They are hashed using the pbkdf2 algorithm before being saved. At login, the entered password is hashed and compared with the stored hash, so even someone with database access cannot read the actual passwords.

9.2 Role-Based Access Control

Each user has a role (admin, donor, or recipient). The application checks the role before allowing access to a page, so a donor cannot open admin pages, and a recipient cannot manage inventory.

9.3 SQL Injection Prevention

Every database query uses parameterized queries (placeholders), so user input can never be executed as SQL.

9.4 Session Management

After a successful login, a secure session keeps the user logged in. Logging out clears the session so the account cannot be accessed by others.

10. Concurrency Control

Concurrency control ensures that the database stays correct even when several actions happen at the same time. The key example in this system is approving blood request.

When an administrator approves a request, two things must happen together: a matching blood unit must be reserved, and the request must be marked approved. If only one of these happened, the data would become inconsistent.

10.1 ACID Properties

- Atomicity — both updates succeed together, or neither does (ROLLBACK on error).
- Consistency — the inventory and request stay logically correct.
- Isolation — FOR UPDATE locks the unit so two admins cannot reserve the same one.
- Durability — once committed, the change is permanent.

11. Testing

The system was tested by performing each operation and confirming the result in the database and on screen.

Test Case	Expected Result	Outcome
Login with correct password	User logged in, redirected to role page	Pass
Login with wrong password	Access denied, error message	Pass
Admin adds a new donor	Donor appears in the donor list	Pass
Admin approves a blood request	Request approved, a unit becomes reserved	Pass
Recipient submits a request	Request appears under 'My Requests' as pending	Pass
Donor books an appointment	Appointment appears in donor and admin views	Pass
Donor tries to open admin page	Access blocked by role check	Pass

13. Conclusion and Future Enhancements

The Blood Donation Management System successfully demonstrates the practical application of database concepts including ER modelling, normalization, indexing, security, and concurrency control. It provides a working, role-based interface for managing blood donation in Sargodha and keeps all data consistent and secure.

Future enhancements could include email or SMS notifications to donors, automatic detection of compatible blood groups, expansion to multiple cities, an online dashboard with charts, and deployment to a live web server so users can access it over the internet.

14. References

Hoffer, J. A., Venkataraman, R., and Topi, H. Modern Database Management. Pearson.

MySQL Documentation — <https://dev.mysql.com/doc/>

Flask Documentation — <https://flask.palletsprojects.com/>